```
        loss.backward()
        optimizer.step()
        cur_state = torch.Tensor(one_hot(arms,env.get_state()))
9
    return np.array(rewards)
```

- ***1*** Gets current state of the environment; converts to PyTorch variable
- ***2*** Runs neural net forward to get reward predictions
- ***3*** Converts reward predictions to probability distribution with softmax
- ***4*** Normalizes distribution to make sure it sums to 1
- ***5*** Chooses new action probabilistically
- ***6*** Takes action, receives reward
- ***7*** Converts PyTorch tensor data to numpy array
- ***8*** Updates one_hot_reward array to use as labeled training data
- ***9*** Updates current environment state

Go ahead and run this function. When we train this network for 5,000 epochs, we can plot the moving average of rewards earned over the training time (see figure 2.7; we omitted the code to produce such a graph). Our neural network indeed learns a fairly good understanding of the relationship between states, actions and rewards for this contextual bandit. The maximum reward payout for any play is 10, and our average is topping off around 8.5, which is close to the mathematical optimum for this particular bandit. Our first deep reinforcement learning algorithm works! Okay, it's not a very deep network, but still!

Figure 2.7. A training graph showing the average rewards for playing the contextual bandit simulator using a two-layer neural network as the action-value function. We can see the average reward rapidly increases during training time, demonstrating our neural network is successfully learning.